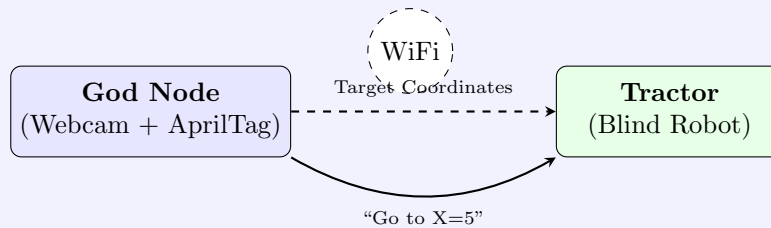


ROS2 Spring Workshop

Lab 5: The Global Eye (Vision, Calibration & Transforms)

Project Context: The God Node Architecture

The Goal: For the final project your robot will use a distributed system. The Camera (Server) stays fixed on the ceiling and tells the Tractor (Client) where it is. The Tractor can use these coordinates to position itself and achieve its goals.



Objective

In previous labs, we relied on `/turtle1/pose` to tell us exactly where the robot was. This was “easy mode”: perfect, noiseless data. In the real world, robots must use sensors to *calculate* where they are.

In this lab, you will build the “eye” of the God Node proposed in your Final Project. You will learn three things that every real robotics vision system needs:

1. How to detect and identify visual markers (AprilTags).
2. How to **calibrate** a camera so the math is actually “accurate”.
3. How difficult it is to make cameras accurate.
4. How to read 3D pose information out of a transform tree.

You may use any USB webcam or your laptop’s integrated camera.

Pre-Lab Concept Primer

1. What is an AprilTag?

An AprilTag is a visual fiducial system (like a QR code for robots). It allows a camera to determine the precise 3D position (XYZ) and orientation (roll-pitch-yaw) of the tag relative to the lens.

Why “36h11”? You will see this code often. AprilTags come in “families”:

- **36:** The tag has a 6x6 bit grid (36 bits of data).
- **h11:** It has a “Hamming distance” of 11. This means the tag is extremely robust; even if 10 bits are covered by mud or glare, the robot can still read it mathematically.

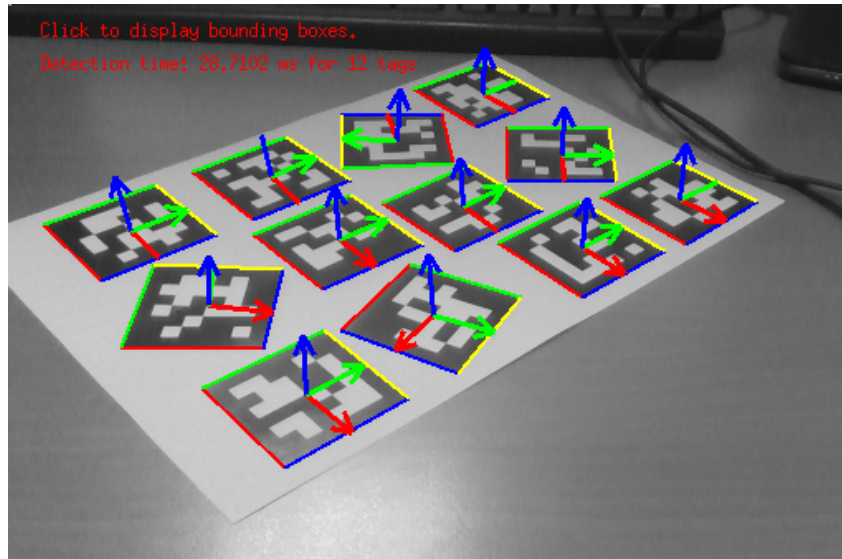


Figure 1: An AprilTag detector at work: each tag is located in the image (coloured outlines) and a full 3D pose is computed for it. The red/green/blue arrows are the tag's X/Y/Z axes, the blue arrow points out of the tag along its surface normal, which is how the camera knows the tag's orientation in space.

2. The Transform Tree (TF)

Standard cameras are “optical frames”:

- **Z-axis:** Points *forward* (out of the lens).
- **X-axis:** Points *right*.
- **Y-axis:** Points *down*.

This is weird! In our map (world frame), **Z** is usually *up*. ROS 2 uses the **TF2 library** to handle this conversion automatically, and AprilTag has a `z_up` parameter that we'll use to match the world convention.

3. Why Calibration Matters (The Pinhole Model)

A camera's math assumes a **pinhole model**: light enters through one point and projects onto a flat sensor. Real lenses bend light. If we don't tell the software exactly how YOUR specific lens bends light, every measurement will be off by 5–20%.

Two numbers matter most:

- **Focal length:** How “zoomed in” the lens is, in pixels.
- **Distortion coefficients:** How the lens curves straight lines.

We find these by showing the camera a **checkerboard of known geometry** from many angles, and letting OpenCV solve for the lens parameters that make all those checkerboard corners line up. This is what you'll do in Part 2.

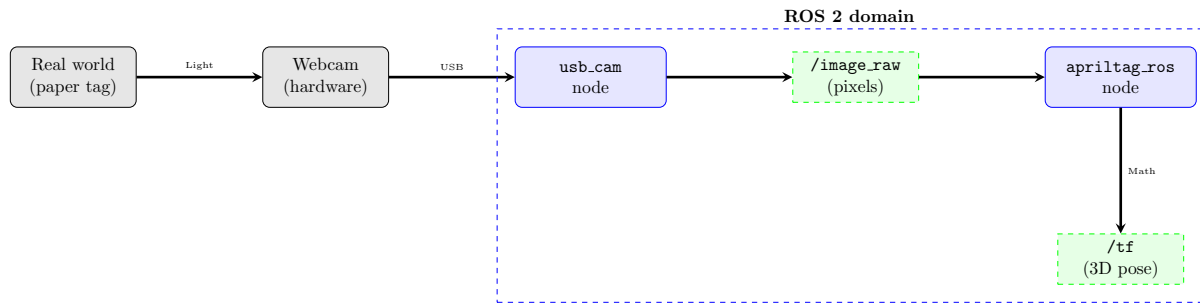


Figure 2: The Vision Pipeline: how reality gets converted into math.

Phase 0: The Hardware Handshake

Before You Open VS Code: Set the Camera Device

Open `.devcontainer/devcontainer.json` in any text editor (Notepad is fine). Find the `runArgs` block and make sure the device line reads:

```
"--device=/dev/video0:/dev/video0",
```

Save the file.

If you opened the project in VS Code before: You also need to delete your old container, or the rebuild will reuse the broken config. In Docker Desktop find `agtech_ros2` and click the trash icon.

Linux users: Your USB camera should already appear as `/dev/video0` automatically. **Skip directly to Part 1.**

macOS Users: Read This Before You Start

Docker Desktop on macOS **does not reliably support USB device passthrough** to containers. This is a hypervisor-level limitation, not something XQuartz or a different webcam can fix. (Docker Desktop 4.35+ added experimental USB/IP support, but macOS lacks a proper USB/IP server, so it's not viable for a 2-hour lab.)

For Labs 5 and 6 you will need to pair with a classmate on Windows or Linux. This is the recommended path. You can still write and review the code on your own laptop; just run the camera parts on theirs.

Optional Mac sanity check: If you want to know whether your specific Docker Desktop version supports USB passthrough (it's evolving), open the Dev Container terminal and run `ls /dev/video*`. If you see `/dev/video0`, congratulations!!! you can do Labs 5 and 6 solo. If you get "No such file or directory," plan to pair up. The check takes 5 seconds.

Docker on Windows cannot see your webcam by default. Windows "hogs" it. You must detach it from Windows and plug it into the Linux kernel (WSL 2).

Do this in PowerShell (Admin)

1. **Install the tool:** `winget install dorssel.usbipd-win` (then restart your computer).
2. **Find your Bus ID:** `usbipd list` (e.g., let's say it is 1-13).
3. **Bind it:** `usbipd bind --busid 1-13`
4. **Attach to Docker:** `usbipd attach --wsl --busid 1-13`

Note: If you get an error, ensure Docker Desktop is running first.

Verification: Inside your VS Code terminal, run `ls -l /dev/video*`. If you see yellow text (`/dev/video0`), you are ready to proceed.

Part 1: Package Setup

Before we configure the camera, we need a home for our vision logic.

1. Go to the source folder:

```
1 cd /workspaces/agtech_ros2/src
2
```

2. Create the package:

We use `ament_python` for this lab because our geofence node (Part 7) is written in Python, and Python packages give us a tidy `setup.py` for registering executables.

```
1 ros2 pkg create --build-type ament_python lab5_vision
2
```

What You'll See After Creation

`ament_python` creates this structure inside `lab5_vision/`:

- `lab5_vision/` ... the inner Python module folder (your `.py` files go here)
- `resource/` ... ROS 2 package marker (don't touch)
- `test/` ... linter tests (don't touch)
- `package.xml` ... ROS package manifest
- `setup.py` ... Python install config (you'll edit this in Part 7)
- `setup.cfg` ... Python build options

Don't be alarmed by the duplicate `lab5_vision` name. The outer one is the package, the inner one is the Python module. That's just how Python packaging works.

3. Create the config folder:

We need a place to store our tag settings.

```
1 mkdir -p /workspaces/agtech_ros2/src/lab5_vision/config
2
```

4. Copy the tag config:

We have provided the `tags.yaml` file for you in the course resources.

```
1 cp /workspaces/agtech_ros2/lab_resources/lab5/tags.yaml \
2   /workspaces/agtech_ros2/src/lab5_vision/config/
3
```

Part 2: Real Camera Calibration

This is where many tutorials cheat. They hand you a “dummy calibration file” and your distance measurements come out 5–20% wrong. We're not doing that. You're going to calibrate your actual camera with a checkerboard, and your measurements will be accurate to 1–2%.

The Calibration Board

Your instructor has printed a **checkerboard calibration target** and distributed one to each group. It is an 8×5 grid of black and white squares. That means 7×4 **internal corners** (the points where 4 squares meet).

Each square should be **50 mm** on a side. **Verify with a ruler** before you calibrate, if your board printed at 95% scale, the math will be wrong by 5%.

Important vocabulary for the command below:

- **Squares** = the black/white tiles (8 wide, 5 tall)
- **Inner corners** = the points where 4 squares meet (7 wide, 4 tall)
- The calibrator asks for **inner corners**, not squares. This trips up everyone.

Step 1: Install the Calibration Tool

```
1 sudo apt install ros-humble-camera-calibration \
2 ros-humble-usb-cam \
3 ros-humble-apriltag-ros
```

Step 2a: Start the Camera

In **Terminal 1**, start the `usb_cam` driver:

```
1 ros2 run usb_cam usb_cam_node_exe --ros-args \
2 -p video_device:=/dev/video0 \
3 -p pixel_format=="mjpeg2rgb"
```

You'll see some warnings about "camera not calibrated". That's fine, we're about to fix that.

Sanity Check the Feed

Before waving a checkerboard around, confirm the camera is pointed where you think and the image looks reasonable. In **Terminal 2**:

```
ros2 run rqt_image_view rqt_image_view
```

Pick `/image_raw` from the topic dropdown. Wave your hand to confirm it's live. If the image is very dark or blown out, fix the lighting now or corner detection will fail in Step 3. Leave the window open during calibration. It doesn't interfere, and it's handy if something looks off.

Step 2b: Fix the Exposure (If Your Image is Dark or Blown Out)

If your `rqt_image_view` feed looks dim and muddy, OR washed-out and overly bright, your camera is stuck on a manual exposure value that doesn't match your lighting. The `usb_cam` node tries to enable auto-exposure on startup, but many webcams expose that control under a different name than the driver expects, and the setting silently fails. You'll see lines like this in the `usb_cam` terminal:

```
[usb_cam]: Setting 'exposure_auto' to 3
unknown control 'exposure_auto'
```

Either extreme will sabotage Step 3. The calibrator can't find checkerboard corners in a dark image, and it can't find them in a blown-out one either (the black squares bleed into the white ones). Let's fix it directly with `v4l2-ctl`, the standard Linux tool for controlling video devices.

TL;DR. Try This First

For most students, these three commands fix the problem outright. Stop `usb_cam`, install `v4l-utils` if needed, then run:

```
sudo apt install v4l-utils # if not installed
v4l2-ctl -d /dev/video0 -c auto_exposure=3
v4l2-ctl -d /dev/video0 -c brightness=128
```

You might need to restart `usb_cam`. If your image now looks normal, you're done! Move on to Step 3.

The full steps and tuning options below are for cases where the quick fix doesn't get you there, or where you want a rock-solid setup for calibration.

1. **Install `v4l2-utils`** (if you don't have it already):

```
1 sudo apt install v4l-utils
2
```

2. **Stop the camera node** (Ctrl+C in Terminal 1). `v4l2-ctl` talks to the camera directly, and only one process can own `/dev/video0` at a time.
3. **List what your camera actually supports:**

```
1 v4l2-ctl -d /dev/video0 --list-ctrls
2
```

Look for `auto_exposure`, `brightness`, and `exposure_time_absolute`. Note their current `value` fields:

- If `auto_exposure` is 1 (Manual Mode) and `brightness` is well below 128 (default) → your image is **too dark**.
- If `auto_exposure` is 1 and `exposure_time_absolute` is near its max → your image is **blown out**.
- If `auto_exposure` is already 3 but the image still looks bad → auto-exposure is enabled but struggling with your lighting; skip to the manual-mode box below.

Will These Commands Work on My Specific Camera?

The *control names* (`auto_exposure`, `brightness`, etc.) and the *mode values* (1=manual, 3=auto) are standardized by the USB Video Class (UVC) spec, which nearly every USB webcam follows. So the commands work the same on a Logitech, a built-in laptop camera, or a generic Amazon webcam.

What *does* differ camera-to-camera is the **range** of each control. Your `exposure_time_absolute` might max out at 1250 while a classmate's goes to 5000. That's why Step 3 (`--list-ctrls`) is non-skippable: it tells you your camera's actual min/max so you can pick sensible values.

4. Turn on auto-exposure and restore defaults:

```
1 v4l2-ctl -d /dev/video0 -c auto_exposure=3
2 v4l2-ctl -d /dev/video0 -c brightness=128
3
```

Mode 3 on most UVC cameras means “Aperture Priority” (auto-exposure on). This fixes most cases in either direction.

5. Restart `usb_cam` (the command from Step 2). Your image should now look properly exposed in `rqt_image_view`.

Prefer Manual Exposure for Calibration

Auto-exposure constantly re-adjusts as you move the checkerboard around, which can confuse the corner detector at edges of the frame. For the cleanest calibration, lock the exposure to a fixed value once you're in a stable lighting environment:

```
v4l2-ctl -d /dev/video0 -c auto_exposure=1
v4l2-ctl -d /dev/video0 -c exposure_time_absolute=400
v4l2-ctl -d /dev/video0 -c brightness=128
```

Mode 1 is manual exposure. Tune `exposure_time_absolute` based on what you're seeing:

- **Image too dark?** Increase the number (try 600, 800, 1000...).
- **Image too bright/washed out?** Decrease it (try 200, 100, 50...).

The valid range varies by camera. Check your `--list-ctrls` output for the **min** and **max**. The checkerboard's black squares should look clearly black, the white squares clearly white, with no blown-out highlights from overhead lighting and no muddy grey patches.

Why Not Just Pass These to `usb_cam`?

`usb_cam` does accept `-p brightness:=128` and similar parameters, but it uses its own control name mapping that doesn't match every camera. Setting controls with `v4l2-ctl` *before* starting `usb_cam` is the most reliable approach because it talks directly to the V4L2 kernel driver. The settings persist until you unplug the camera or reboot the node.

Step 3: Run the Calibrator

In Terminal 2:

```
1 ros2 run camera_calibration cameracalibrator \  
2   --size 7x4 --square 0.050 \  
3   --ros-args -r image:=/image_raw -r camera_info:=/camera_info
```

Decoding the arguments:

- `--size 7x4` is **inner corners**, not squares. Your board has 7×4 inner corners.
- `--square 0.050` is the square edge length in **meters** (50 mm).

Step 4: Collect Calibration Data

A GUI window appears showing your camera feed. On the right, you'll see four progress bars: **X**, **Y**, **Size**, and **Skew**.

Your job: hold the checkerboard in front of the camera and move it around until all four bars fill up and the **CALIBRATE** button lights up green.

Tips for fast calibration:

- **X**: Move the board left and right across the whole frame.
- **Y**: Move it up and down.
- **Size**: Move it close to the camera (filling most of the frame) and far (small in the frame).
- **Skew**: Tilt the board at various angles: forward, backward, sideways. This is usually the hardest to fill.
- Good lighting helps the detector lock onto corners. Avoid harsh shadows on the board.

You'll need about 40 detected views. This takes 2–3 minutes.

Step 5: Commit the Calibration

Once **CALIBRATE** is green, click it. The tool will think for 30–60 seconds (doing math), then the **SAVE** and **COMMIT** buttons light up.

Click **SAVE** (writes a tarball to `/tmp`), then **COMMIT** (publishes the calibration to the camera node).

Step 6: Install the Calibration Permanently

The calibration is now published, but lives only in memory. Close the calibrator window. The tarball at `/tmp/calibrationdata.tar.gz` contains a file called `ost.yaml`: that's your calibration.

Extract it and install:

```
1 # Unpack the calibration tarball  
2 cd /tmp  
3 tar -xzf calibrationdata.tar.gz  
4  
5 # Make the config folder in the right place  
6 mkdir -p ~/.ros/camera_info  
7  
8 # Rename ost.yaml to match the camera_name (usb_cam's default is "default_cam")  
9 # Open ost.yaml and find the line "camera_name: narrow_stereo"  
10 # Change it to: camera_name: default_cam  
11 # Then save as ~/.ros/camera_info/default_cam.yaml:  
12 cp /tmp/ost.yaml ~/.ros/camera_info/default_cam.yaml
```

The Name-Matching Trap

The calibrator saves the file with `camera_name: narrow_stereo` by default (a historical default from a stereo camera rig). The `usb_cam` node looks for a file whose `camera_name` matches `default_cam`. **If these do not match, the driver silently ignores your calibration** and you'll wonder why nothing changed.

Open `/.ros/camera_info/default_cam.yaml` in VS Code and change:

```
camera_name: narrow_stereo
```

to:

```
camera_name: default_cam
```

Save the file.

Step 7: Sanity-Check Your Calibration

Open your `default_cam.yaml` and look at the `camera_matrix`. The data array contains 9 numbers in row-major order:

```
camera_matrix:
  data: [fx,  0,  cx,
         0, fy,  cy,
         0,  0,  1]
```

- `fx`, `fy` = focal length in pixels (1st and 5th values)
- `cx`, `cy` = optical center in pixels (3rd and 6th values)

Important: Focal length in pixels *scales with image resolution*. A 1080p camera will report `fx` around 900, a 720p camera around 600, a 480p camera around 400. So don't compare your numbers to a classmate's unless you're both running at the same resolution. Check the `image_width` and `image_height` lines at the top of the file to know what to expect.

The three sanity checks (these are what matter):

1. **Focal lengths agree:** `fx` and `fy` should be within 5% of each other. A good calibration usually gets them within 1%. If they disagree by more than 5%, your calibration captured too few views or poor pose variety: recalibrate.
2. **Optical center is roughly centered:** `cx` should be near `image_width / 2`, and `cy` near `image_height / 2`. Being off by 10–30 pixels is normal (sensor isn't perfectly aligned with the lens during manufacturing). Being off by 100+ pixels suggests a bad calibration.
3. **Distortion is small:** The first two distortion coefficients (`k1`, `k2`) should be well under 0.5 in absolute value. If `k1` is greater than 1.0, something went wrong: likely your checkerboard wasn't flat, or you only captured views from one part of the frame.

Example: A Good 640×480 Calibration

For reference, a clean calibration of a typical 480p USB webcam might look like this:

```
image_width: 640
image_height: 480
camera_matrix:
  data: [410.18, 0,      302.34,
         0,      410.55, 249.78,
         0,      0,      1]
distortion_coefficients:
  data: [0.015, -0.030, 0.003, -0.005, 0.000]
```

Reading this: `fx` and `fy` agree to within 0.1%, the optical center (302, 250) sits close to the geometric center (320, 240), and `k1`=0.015 is tiny: this lens barely distorts. **If your numbers look proportional to this for your resolution, you're in great shape.**

Part 3: Running the Camera with Your Calibration

Now restart the camera, this time pointing it at your calibration file:

```
1 ros2 run usb_cam usb_cam_node_exe --ros-args \  
2   -p video_device:=/dev/video0 \  
3   -p pixel_format:="mjpeg2rgb" \  
4   -p camera_name:=default_cam \  
5   -p camera_info_url:="file:///root/.ros/camera_info/default_cam.yaml"
```

You will want to apply the same brightness adjustments as before. If you want to just use auto-exposure you can use the the following,

```
1 v4l2-ctl -d /dev/video0 -c auto_exposure=3  
2 v4l2-ctl -d /dev/video0 -c brightness=128  
3
```

This time, no “camera not calibrated” warning.

Syntax Spotlight: Anatomy of a ROS 2 Command

Let’s decode this command:

- **ros2 run**: The verb. We’re running a program.
- **usb_cam**: The **package** (the library folder).
- **usb_cam_node_exe**: The **executable** (the actual script inside the folder).
- **--ros-args**: *“Everything after this is for ROS, not Linux.”*
- **-p**: Sets a **parameter** (configuration variable).
- **pixel_format:="mjpeg2rgb"**: We force the node to use MJPEG compression to save USB bandwidth, then unzip to RGB for ROS.

Part 4: The Detector (AprilTag Node)

Now we convert “pixels” into “pose”.

Critical: Do NOT use your phone screen!

You must use a **printed paper tag**. If you are attending this class in person I will provide you with one.

- **Why?** Computer vision algorithms look for sharp contrast. A phone screen emits light and has a glass surface that reflects your room lights (specular reflection). This blinds the camera.
- **Get the tag:** <https://chaitanyantr.github.io/apriltag.html>
- **Instruction:** Select family **tag36h11** and ID **0**.

Measure Your Tag’s Black Square

The **size** parameter in **tags.yaml** is the **edge length of the BLACK SQUARE in meters**, not the whole printed image, not the page, just the black part.

Grab a ruler. Measure the black square on your printed tag. A typical print on letter/A4 paper is around 12 cm (0.12m), but yours might be different.

Open **tags.yaml** and set **size:** to the value you measured. If you don’t, your distance estimates will be wrong by the same ratio.

1. **Edit your config file:** Open **tags.yaml**. We’ve added performance parameters for you to see.

```
1 apriltag:  
2   ros__parameters:  
3     family: 36h11  
4     size: 0.12           # EDGE LENGTH OF THE BLACK SQUARE (measure it!)  
5     max_hamming: 0  
6     z_up: true          # Report tag Z as depth into the scene  
7     # --- PARAMETERS TO TUNE ---
```

```

8   decimate: 2.0          # 1.0 = full res, 2.0 = half res (faster)
9   threads: 4            # Use more CPU cores
10  refine_edges: 0        # 1 = precise, 0 = fast
11

```

What does `z_up: true` actually do?

Without `z_up: true`, the detector reports tag pose in the camera’s *optical* frame where Z points forward out of the lens. Distance to the tag ends up on the **Z axis**, which is counterintuitive when visualized in RViz.

With `z_up: true`, the reported pose is rotated so that Z points up (matching the world convention). Depth into the scene now shows up on the axis your brain expects. Your “Reality Gap” challenge at the end measures distance along the camera’s optical Z axis either way. With `z_up: true`, the numbers match the way humans naturally think about space.

2. Run the node:

```

1  ros2 run apriltag_ros apriltag_node --ros-args \
2    -r image_rect:=/image_raw \
3    -r camera_info:=/camera_info \
4    --params-file /workspaces/agtech_ros2/src/lab5_vision/config/tags.yaml
5

```

About That `image_rect` Remap

We map the topic `image_rect` (which suggests a pre-rectified image) to `/image_raw` (which is NOT pre-rectified). This works because AprilTag applies distortion correction internally using `camera_info`.

In Lab 9 we’ll add a proper `rectify_node` in between for better accuracy at the edges of the frame. For today, the internal correction is fine.

Part 5: Verification

Before opening RViz, do a quick terminal check. Hold up your tag to the camera and type in a new terminal:

```

1  ros2 topic echo /tf --once

```

You should see a transform with `child_frame_id: tag36h11:0`. **If you see this, detection is working.** RViz is optional confirmation, the detector doesn’t need it.

RViz Visualization

1. Open RViz: `rviz2`

2. The “ghost frame” fix: In the top-left “Global Options”, the **Fixed Frame** dropdown might be empty because ROS hasn’t seen the camera yet. **Manually type:** `default_cam` and hit Enter.

3. Add the camera feed:

- Click **Add** (bottom-left) and select **Camera** (not just “Image”).
- Set the **Topic** to `/image_raw`.
- *Why? The “Camera” display type automatically overlays 3D content on top of the 2D video feed.*

4. Add the TF display (this is what shows the axis arrows):

- Click **Add** again and select **TF**.
- You should immediately see two coordinate frames listed under the new TF display: `default_cam` (your camera) and `tag36h11:0` (your tag).

- The red/green/blue axis arrows for the tag will now appear overlaid on top of the video feed, attached to the physical tag.

Don't See the Tag Frame?

If the TF display only shows `default_cam` and not `tag36h11:0`, the AprilTag detector isn't seeing your tag. Common causes:

- **Part of the tag is being covered by your hand or fingers.** AprilTags are very sensitive to missing information. The detector has to find the full black border to even attempt decoding. The “Hamming distance 11” robustness only applies *after* the tag has been located; it can't recover a tag whose outline is broken.
- Tag is too far away, too close, or partially out of frame.
- Lighting is poor, harsh shadows or glare on the tag.
- You're holding the tag at too steep an angle.
- The tag is bent or the paper is curled (must be flat).

Verify detection from a terminal: `ros2 topic echo /tf --once` should print a transform with `child_frame_id: tag36h11:0`. If that command hangs, the detector isn't detecting.

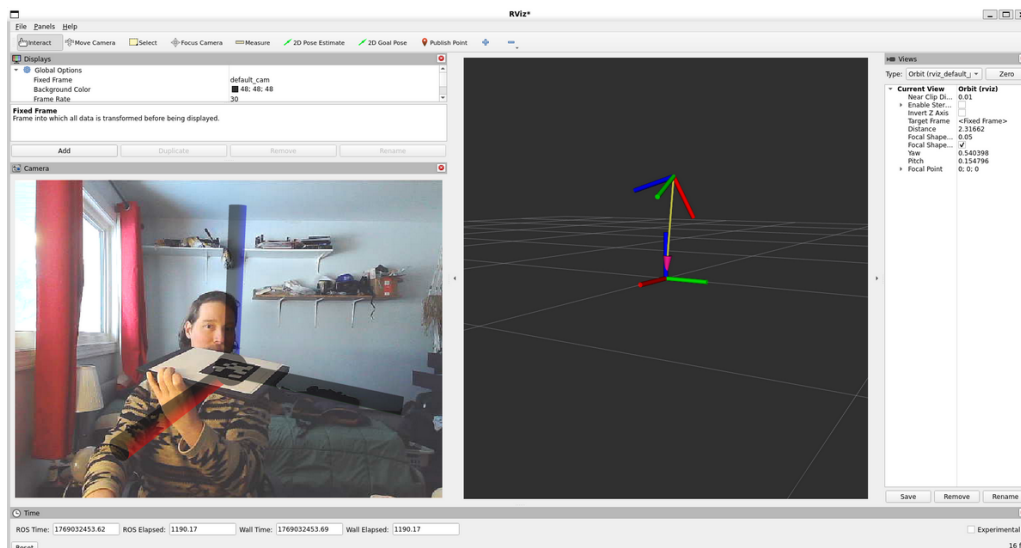


Figure 3: Expected result: the TF axes should appear firmly attached to the physical tag in the video feed.

Part 6: The Reality Gap (and Why Your Calibration Matters)

Robots trust their sensors, but sensors lie. You already did the work to shrink that lie: real calibration. Let's measure how well it paid off.

1. Place your AprilTag flat on a desk.
2. Use a tape measure to position the camera **exactly 50 cm (0.50 m)** from the tag. Measure from the front of the lens to the centre of the tag.
3. In RViz, read the tag's reported position. Or in a terminal:

```
1 ros2 topic echo /tf --once
2
```

Look for translation: `z: ...` (with `z.up: true`, this is the depth into the scene).

4. Compute your percent error:

$$\text{Error}\% = \frac{|\text{Measured} - 0.50|}{0.50} \times 100\%$$

5. Now try 1.0 m and 1.5 m. Record all three.

Interpreting Your Error

- **Under 2%:** Excellent. Your calibration is dialed in. You're getting accuracy that rivals commercial indoor positioning systems.
- **2–5%:** Good. Normal for this class of camera. Slight misalignment in your calibration data or tag measurement.
- **5–10%:** Usable but worth investigating. Most likely your tag **size** parameter is off; re-measure the black square.
- **Over 10%:** Something's wrong. Likely suspects: tag size way off, or your calibration used too few views / poor pose variety. Consider recalibrating.

Remember: Before you calibrated, a dummy calibration would have produced 10–20% errors routinely. You “fixed” that by doing this properly.

Pro Tip: Is Your Laptop Lagging?

The detector math is heavy. Here's how to reduce load:

1. In `tags.yaml`, set `decimate: 2.0`. This halves the resolution the detector uses. About 4x faster but might miss small tags.
2. **Close RViz.** RViz rendering can eat 30% of a CPU core. The detector itself runs at the same rate either way. RViz is just a viewer. If you need to verify, use `ros2 topic echo /tf --once` in a terminal.
3. Increase `threads`: if your CPU has 8+ cores. Default 4 is fine for most laptops.

Cleanup Tip

If `Ctrl+C` on `usb_cam` hangs or leaves the device locked, wait 5 seconds for the kernel to release it. If that fails on Linux:

```
fuser -k /dev/video0
```

This force-releases any process still holding the camera.

Part 7: Programming Challenge, The Geofence Sentinel

Scenario: Protecting the Crop Zone

In our God Node architecture, the overhead camera is responsible for monitoring the entire field. Your tractor is navigating, but what if a motor controller fails and it veers off course?

Your task is to complete a custom ROS 2 Python node that acts as a **Geofence Sentinel**. It must listen to the spatial relationship between the camera (`default_cam`) and the tractor (`tag36h11:0`), and warn the system if the tractor leaves a defined safe radius.

The Specifications

I have provided a skeleton script to handle the complex `tf2_ros` listener setup so you can focus entirely on the spatial logic.

1. **Copy the skeleton script:** Navigate to your package and copy the starter file:

```

1 cp /workspaces/agtech_ros2/lab_resources/lab5/geofence_sentinel.py \
2   /workspaces/agtech_ros2/src/lab5_vision/lab5_vision/
3

```

2. **Calculate Planar Distance:** Open the script and find TODO 1. The script already extracts X and Y . Calculate the Euclidean distance of the tractor from the centre of the camera's field of view (ignoring Z depth for now):

$$r = \sqrt{x^2 + y^2}$$

3. **Enforce the Geofence:** Find TODO 2. Assume the safe operating zone is a circle with a radius of **0.40 metres** directly beneath the camera.
4. **Log the Behaviour:**
 - If $r \leq 0.40$, use `get_logger().info()` to print: *"Tractor in safe zone. Distance: [r] m"*
 - If $r > 0.40$, use `get_logger().warn()` to print: *"CRITICAL: Tractor out of bounds!"*

Build Faster: Only Rebuild Your Package

The default `colcon build` compiles every package in your workspace; slow and unnecessary when you're only changing one. Use the `--packages-select` flag to rebuild just `lab5_vision`:

```

cd /workspaces/agtech_ros2
colcon build --packages-select lab5_vision --symlink-install
source install/setup.bash

```

What the flags do:

- `--packages-select lab5_vision ...` build only this one package, ignore the others.
- `--symlink-install ...` symlink your Python files into `install/` instead of copying. After this, edits to `geofence_sentinel.py` take effect *immediately* without rebuilding. You only need to rebuild again if you edit `setup.py` or add new files.

Registration Reminder

Don't forget! You must add `'geofence_sentinel = lab5_vision.geofence_sentinel:main'` to the `console_scripts` array in your `setup.py` file and rebuild the workspace before you can run it.

Testing the Sentinel

Run your vision pipeline (USB cam and AprilTag nodes). Then, run your Sentinel node. Move the physical AprilTag across the perpendicular plane to the camera. Watch your terminal output change colour from standard white/green `INFO` messages to yellow `WARN` messages as it crosses the invisible 40 cm boundary.

Deliverables

Submit a single PDF containing:

1. **Calibration proof:** Your `default_cam.yaml` contents. Specifically flag your focal length values (f_x , f_y), your k_1 distortion coefficient, and whether f_x and f_y are within 5% of each other.
2. **Detection screenshot:** RViz showing the TF axes firmly attached to your physical tag.
3. **Reality Gap measurements:** A table with three rows (50 cm, 1 m, 1.5 m), showing physical distance vs. reported Z vs. percent error. Include a 3–5 sentence analysis of the residual error.
4. **Geofence Proof:** A screenshot of your terminal showing the Sentinel node logging both the safe zone status and the out-of-bounds warning as you moved the tag.